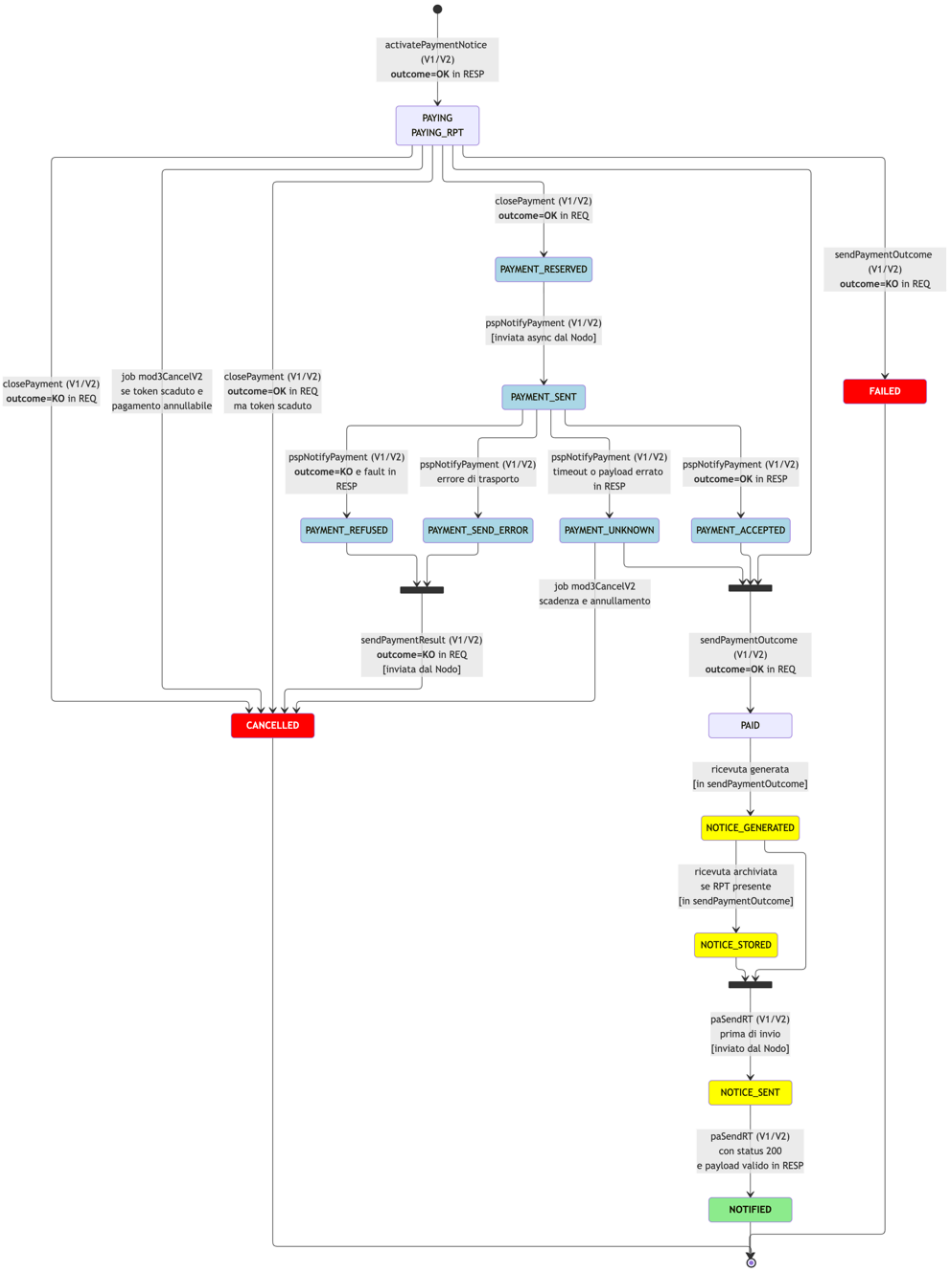
Cambiamento di stati

Entità “Payment”
Entità “Position”
Entità “RPT”
Entità “RT”

Entità “Payment”

L’entità **payment** rappresenta un tentativo di pagamento concreto effettuato da un cittadino, identificato univocamente da un **payment token**.



Codice Mermaid

```
---
config:
  theme: default
  look: classic
---

stateDiagram-v2

    classDef errorState color:white,font-weight:bold,fill:red
    classDef successState font-weight:bold,fill:lightgreen
    classDef nmuFlowState fill:lightblue
    classDef receiptState fill:yellow

    state "PAYING<br/>PAYING_RPT" AS PAYING
```

```
    [*] --> PAYING: activatePaymentNotice (V1/V2)<br/>**outcome=OK** in RESP

    PAYING --> CANCELLED: closePayment (V1/V2)<br/>**outcome=KO** in REQ
    PAYING --> PAYMENT_RESERVED:::nmuFlowState: closePayment (V1/V2)<br/>**outcome=OK** in REQ

    PAYMENT_RESERVED --> PAYMENT_SENT:::nmuFlowState: pspNotifyPayment (V1/V2)<br>[inviata async dal Nodo]

    PAYMENT_SENT --> PAYMENT_ACCEPTED:::nmuFlowState: pspNotifyPayment (V1/V2) <br/>**outcome=OK** in RESP
    PAYMENT_SENT --> PAYMENT_REFUSED:::nmuFlowState: pspNotifyPayment (V1/V2) <br/>**outcome=KO** e fault in RESP
    PAYMENT_SENT --> PAYMENT_UNKNOWN:::nmuFlowState: pspNotifyPayment (V1/V2)<br/>timeout o payload errato in RESP
    PAYMENT_SENT --> PAYMENT_SEND_ERROR:::nmuFlowState: pspNotifyPayment (V1/V2)<br/>errore di trasporto

    PAYING --> FAILED: sendPaymentOutcome (V1/V2)<br/>**outcome=KO** in REQ

    state spr_ok_after_notify_error <<join>>
    PAYING --> spr_ok_after_notify_error
    PAYMENT_ACCEPTED --> spr_ok_after_notify_error
    PAYMENT_UNKNOWN --> spr_ok_after_notify_error
    spr_ok_after_notify_error --> PAID: sendPaymentOutcome (V1/V2)<br/>**outcome=OK** in REQ

    PAYMENT_UNKNOWN --> CANCELLED: job mod3CancelV2<br/>scadenza e annullamento

    state spr_ko_after_notify_error <<join>>
    PAYMENT_REFUSED --> spr_ko_after_notify_error
    PAYMENT_SEND_ERROR --> spr_ko_after_notify_error
    spr_ko_after_notify_error --> CANCELLED: sendPaymentResult (V1/V2)<br/>**outcome=KO** in REQ<br/>[inviata dal Nodo]

    PAYING --> CANCELLED: job mod3CancelV2<br/>se token scaduto e<br/>pagamento annullabile
    PAYING --> CANCELLED: closePayment (V1/V2)<br/>**outcome=OK** in REQ<br/>ma token scaduto

    PAID --> NOTICE_GENERATED:::receiptState: ricevuta generata<br/>[in sendPaymentOutcome]
    NOTICE_GENERATED --> NOTICE_STORED:::receiptState: ricevuta archiviata<br/>se RPT presente<br/>[in sendPaymentOutcome]

    state pasendrt_sending <<join>>
    NOTICE_GENERATED --> pasendrt_sending
    NOTICE_STORED --> pasendrt_sending
    pasendrt_sending --> NOTICE_SENT:::receiptState: paSendRT (V1/V2)<br/>prima di invio<br/>[inviato dal Nodo]

    NOTICE_SENT --> NOTIFIED: paSendRT (V1/V2)<br/>con status 200<br/>e payload valido in RESP

    CANCELLED:::errorState --> [*]
    FAILED:::errorState --> [*]
    NOTIFIED:::successState --> [*]

```

Gli stati `PAYING` e `PAYING_RPT` indicano che il tentativo di pagamento è in corso.

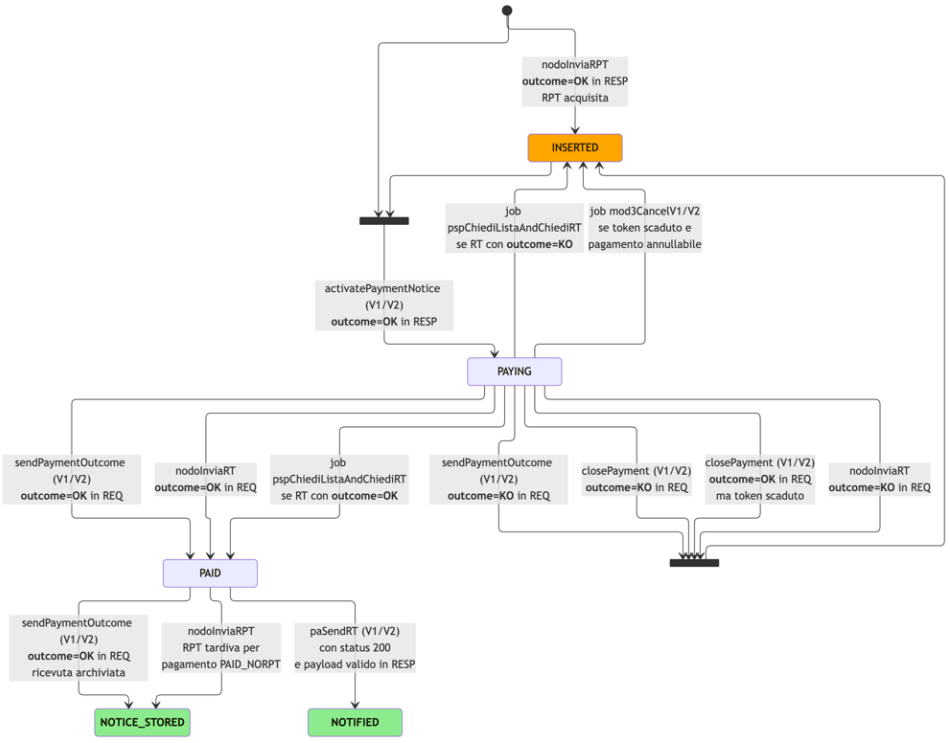
Gli stati `PAYMENT_SENT`, `PAYMENT_ACCEPTED`, `PAYMENT_REFUSED`, `PAYMENT_UNKNOWN` e `PAYMENT_SEND_ERROR` descrivono l'invio e l'esito della notifica al PSP da parte del Nodo. Lo stato `PAYMENT_ACCEPTED` non è ancora lo stato finale: il PSP deve inviare al Nodo una successiva `sendPaymentOutcome` (V1 o V2) con esito OK affinché il pagamento diventi `PAID`. Il Nodo aggiorna lo stato del tentativo appena dopo l'invio della richiesta `pspNotifyPayment` (che viene gestita dal Nodo in asincrono aprendo un nuovo *attore* async che ne gestisce lo stato), prima di restituire la risposta al PSP.

Una `sendPaymentOutcome` inviata con esito KO porta il tentativo ad uno stato `FAILED`, mentre un annullamento o un errore in risposta alla `pspNotifyPayment` può portarlo a `CANCELLED`. La `sendPaymentResult` notifica l'esito ad eCommerce e, nel caso della versione V2, porta direttamente allo stato `CANCELED`.

Dopo il cambio di stato in `PAID`, il Nodo genera e salva la ricevuta durante `sendPaymentOutcome` e, se presente la RPT, la archivia prima di rispondere al PSP. Solo dopo la risposta, il Nodo invia separatamente la ricevuta all'EC tramite una `paSendRT` (V1 o V2): registra `NOTICE_SENT` prima dell'invio e `NOTIFIED` dopo una risposta HTTP 200 con payload valido. Lo stato del pagamento diventa `NOTIFIED` solo quando tutti i destinatari richiesti sono stati notificati.

Entità "Position"

L'entità position rappresenta la posizione debitoria dell'Ente Creditore, gestita dal Nodo dei Pagamenti durante un tentativo di pagamento, identificato univocamente da un **numero avviso** e dal **codice fiscale dell'Ente**.



Codice Mermaid

```
---
config:
  theme: default
  look: classic
---
stateDiagram-v2

    classDef successState font-weight:bold,fill:lightgreen
    classDef oldFlowState fill:orange
    classDef receiptState fill:yellow

    [*] --> INSERTED:::oldFlowState: nodoInviaRPT<br/>**outcome=OK** in RESP<br/>RPT acquisita

    state activate_to_paying <<join>>
    [*] --> activate_to_paying
    INSERTED --> activate_to_paying
    activate_to_paying --> PAYING: activatePaymentNotice (V1/V2)<br/>**outcome=OK** in RESP

    PAYING --> PAID: sendPaymentOutcome (V1/V2)<br/>**outcome=OK** in REQ
    PAYING --> PAID: nodoInviaRT<br/>**outcome=OK** in REQ
    PAYING --> PAID: job pspChiediListaAndChiediRT<br/>se RT con **outcome=OK**


    state paying_to_inserted <<join>>
    PAYING --> paying_to_inserted: sendPaymentOutcome (V1/V2)<br/>**outcome=KO** in REQ
    PAYING --> paying_to_inserted: closePayment (V1/V2)<br/>**outcome=KO** in REQ
    PAYING --> paying_to_inserted: closePayment (V1/V2)<br/>**outcome=OK** in REQ<br/>ma token scaduto
    PAYING --> paying_to_inserted: nodoInviaRT<br/>**outcome=KO** in REQ
    PAYING --> INSERTED: job pspChiediListaAndChiediRT<br/>se RT con **outcome=KO**
    PAYING --> INSERTED: job mod3CancelV1/V2<br/>se token scaduto e<br/>pagamento annullabile
    paying_to_inserted --> INSERTED

    PAID --> NOTICE_STORED:::successState: sendPaymentOutcome (V1/V2)<br/>**outcome=OK** in REQ<br/>ricevuta archiviata
    PAID --> NOTICE_STORED: nodoInviaRPT<br/>RPT tardiva per<br/>pagamento PAID_NORPT

    PAID --> NOTIFIED:::successState: paSendRT (V1/V2)<br/>con status 200<br/>e payload valido in RESP
```

Gli stati associati inizialmente ad una posizione debitoria sono:

- `INSERTED` → indica che l'avviso è pagabile
- `PAYING` → indica che è in corso un tentativo di pagamento

Il Nodo può creare la posizione direttamente in `PAYING` durante una `activatePaymentNotice` (V1 o V2), senza necessariamente avere un precedente stato `INSERTED`.

Quando il Nodo riceve dal PSP una `sendPaymentOutcome` (V1 o V2) con `outcome=OK`, il Nodo registra il pagamento come `PAID`. Se la richiesta è collegata a una RPT, nella stessa elaborazione genera e archivia la ricevuta: per questo la posizione passa da `PAID` a `NOTICE_STORED`, prima che il Nodo risponda al PSP.

Dopo aver risposto al PSP, il Nodo invia la ricevuta all'EC con una `paSendRT` (V1 o V2). Prima dell'invio registra `NOTICE_SENT` mentre dopo una risposta HTTP 200 con payload valido, registra il destinatario come `NOTIFIED`. Per questa transizione non è necessario che la risposta dell'EC abbia `outcome=OK`. Il pagamento passa a `NOTIFIED` quando tutti i destinatari richiesti sono stati notificati. La posizione passa a `NOTIFIED` se solo se, al momento dell'aggiornamento, è ancora in stato `PAID` e ricorrono le condizioni previste per il destinatario. Se la posizione è già `NOTICE_STORED`, resta in quello stato: la consegna può aggiornare le informazioni memorizzate in tabella per il destinatario e il tentativo di pagamento, ma non la posizione.

Un KO della `sendPaymentOutcome` riporta la posizione a `INSERTED` solo se il Nodo accetta l'esito per il tipo e lo stato del pagamento: ad esempio, il KO è rifiutato per i pagamenti eCommerce chiusi con `closePaymentV2`. La stessa riapertura della posizione può essere eseguita da una `closePayment` o dai job di annullamento, se lo stesso è ancora annullabile.

Entità "RPT"

🚧

Entità "RT"

🚧
